

Week 03

SQL data definition

Cảm ơn thầy Trần Duy Quang đã cung cấp template cho môn học
Cảm ơn thầy Hồ Tuấn Thanh và thầy Đỗ Nguyên Kha đã xây dựng nội dung thực hành



Lecturer: **TS. Lê Thị Nhân.**
Lab TA: **ThS. Nguyễn Hải Đăng.**



1

Notes

Put **ALL** SQL statements to **ONE** text (SQL) document names student ID 1651xxx_W03_yy.sql (yy is the number of your problems solved):

- Comment your Student ID and Full name on top of file
- Run **ALL lines** in file without any error
- USE the **CORRECT** database

2

Create table

```
CREATE TABLE [table name]
(
    [attribute 1] [data type 1] [modifier 1] [modifier 2]...,
    [attribute 2] [data type 2] [modifier 1] [modifier 2]...,
    ...
    PRIMARY KEY([list of attributes]),
    UNIQUE([list of attributes]),
    FOREIGN KEY([list of attributes]) REFERENCES [other table] ([list of attributes
    in primary key])
);
```

Example:

```
CREATE TABLE Departments
(
    ID INT NOT NULL,
    Name VARCHAR(50) NOT NULL,
    PRIMARY KEY(ID)
);

CREATE TABLE Employees
(
    -- NOT NULL: Not accept null value. Default is nullable
    ID INT NOT NULL,
    Code CHAR(9) NOT NULL,
    FirstName VARCHAR(50) NOT NULL,
    LastName VARCHAR(50) NOT NULL,
    Birthday DATE,
    -- DEFAULT [value]: Value is used if new record has no value for this attribute
    DepartmentID INT NOT NULL DEFAULT 1,
```

```
-- CHECK ([condition]): Value must meet the condition (or combination of
conditions) in the clause
Sex CHAR(1) NOT NULL CHECK (Sex = 'M' OR Sex = 'L' OR Sex = 'N'),
DepartmentID INT,
-- PRIMARY KEY([list of columns]): One table SHOULD have one (and only one)
primary key but primary key may include one or more columns. The columns
in the primary key must not be null (NOT NULL). Primary key cannot be duplicated
PRIMARY KEY(ID),
-- UNIQUE([list of columns]): Unique keys cannot be duplicated
UNIQUE(Code),
-- FOREIGN KEY: Foreign keys must refer to other tables' primary key. Tforeign
key must have the same number of attributes, same data type, and same order of
attributes. Foreign key values may be null (if the attribute id nullable). If not
null, the value must exist in the reference table
FOREIGN KEY(DepartmentID) REFERENCES Departments (ID)
);
```

The primary keys, uniques, and foreign keys can be declared as an attribute modifier if they only include only 1 column.

```
CREATE TABLE Employees
(
  -- NOT NULL is not mandatory in this case
  ID INT NOT NULL PRIMARY KEY,
  Code CHAR(9) NOT NULL UNIQUE,
  ...
  DepartmentID INT REFERENCES Departments (ID),
);
```

Change table definition

```
ALTER TABLE
ADD [attribute] [data type] [modifier 1]...
DROP COLUMN [attribute] [modifier 1]...
```

```
ALTER COLUMN [attribute] [modifer 1]...  
ADD CONSTRAINT...  
DROP CONSTRAINT...
```

Example:

```
ALTER TABLE  
ADD CONSTRAINT FK_EMPLOYEE_BELONGS_TO_DEPARTMENT  
FOREIGN KEY(DepartmentID) REFERENCES Departments (ID)  
  
ALTER TABLE  
DROP CONSTRAINT FK_EMPLOYEE_BELONGS_TO_DEPARTMENT
```

The constrain name for a foreign key should be prefixed by **FK_**. In case the tables reference each other or self-reference hence the foreign keys must be declared later AFTER all the tables were created.

Update data

```
UPDATE [table name] SET [attribute 1] = [value 1],... WHERE [condition]
```

Example:

```
UPDATE Employees SET DepartmentID = 1 WHERE ID = 10
```

In case the tables reference each other or self-reference, the value of the foreign key may be set to null and then update to the desired value.

Simple data query

```
SELECT [list of columns]  
FROM [list of tables]  
WHERE [condition]  
ORDER BY [attribute 1 [DESC]]
```

Example:

```
-- Simple condition  
SELECT ID, Code, FirstName, LastName FROM Employees WHERE Code = '123456789'
```

```
-- Simple sorting
SELECT * FROM Employees ORDER BY DepartmentID DESC

-- Multi-table select
SELECT d.ID, d.Name, e.FirstName, e.LastName
FROM Employees e, Departments d
WHERE d.ID = e.DepartmentID

-- Or using JOIN
SELECT d.ID, d.Name, e.FirstName, e.LastName
FROM Employees e
JOIN Departments d ON d.ID = e.DepartmentID
```

Composite primary key

The primary key of a table may include more than 1 column – a composite primary key.

```
CREATE TABLE Products
(
  ID INT NOT NULL PRIMARY KEY,
  Name VARCHAR(255) NOT NULL,
  Price INT NOT NULL,
);

CREATE TABLE Orders
(
  ID INT NOT NULL PRIMARY KEY,
  Customer VARCHAR(255) NOT NULL,
);

CREATE TABLE OrderDetails
```

```
(  
  OrderId INT NOT NULL REFERENCES Orders (ID),  
  ProductId INT NOT NULL REFERENCES Products (ID),  
  Quantity INT NOT NULL,  
  PRIMARY KEY(OrderId, ProductId),  
);
```

Foreign key references to composite key

The foreign key reference to a composite primary key must have the same order and data types.

```
CREATE TABLE OrderDetailNotes  
(  
  ID INT NOT NULL PRIMARY KEY,  
  OrderId INT,  
  ProductId INT,  
  Note VARCHAR(255) NOT NULL,  
  FOREIGN KEY(OrderId, ProductId) REFERENCES OrderDetails (OrderId, ProductId)  
);
```

Self-reference table

A column in a table may refer to itself primary key.

```
CREATE TABLE Employees  
(  
  ID INT NOT NULL PRIMARY KEY,  
  Name VARCHAR(255) NOT NULL,  
  ManagerID INT REFERENCES Employees (ID),  
);
```

3 Lab

A: YY: 08

Database name: Company (Ref: Fundamentals of database systems)

Tables:

- Employees:
 - ID: int, not null, primary
 - Code: char(9), not null, unique
 - FirstName: nvarchar(50), not null
 - LastName: nvarchar(50), not null
 - Birthday: date
 - Sex: int, not null ('F', 'M', 'N'). N is non-disclosure
 - Address: varchar(255)
 - **ReportID: int, report to employee id, self-reference**
 - DepartmentID: int, reference to the ID of Departments table
- Departments:
 - ID: int, not null, primary
 - Name: nvarchar(50), not null
 - ManagerID: int, reference to the ID of Employees table
- DepartmentLocations: (*)
 - DepartmentID: int, not null, primary, reference to the ID of Departments table
 - Location: nvarchar(50), not null, primary

(*) Only one (or no) department in a location

Create SQL file with requirements:

1. Create database
2. Create tables
3. Add sample data

Employees

ID	Code	FirstName	LastName	Birthday	Sex	Address	ReportID	DepartmentID
1	123456789	John	Smith	1965-01-09	M	731 Fondren, Houston, TX	2	5
2	333445555	Franklin	Wong	1955-12-08	M	638 Voss, Houston, TX	8	5
3	999887777	Alicia	Zelaya	1968-01-19	F	3321 Castle, Spring, TX	4	4
4	987654321	Jennifer	Wallace	1941-06-20	F	291 Berry, Bellaire, TX	8	4
5	666884444	Ramesh	Narayan	1962-09-15	M	975 Fire Oak, Humble, TX	2	5
6	453453453	Joyce	English	1972-07-31	F	5631 Rice, Houston, TX	2	5
7	987987987	Ahmad	Jabbar	1969-03-29	M	980 Dallas, Houston, TX	4	4

8	888665555	James	Borg	1937-11-10	M	450 Stone, Houston, TX	NULL	1
---	-----------	-------	------	------------	---	---------------------------	------	---

Departments

ID	Name	ManagerID
1	Research	2
2	Headquarters	4
3	Headquarters	8

DepartmentLocations

DepartmentID	Location
1	Houston
4	Stafford
5	Bellaire
5	Sugarland
5	Houston

4. List all male employees (ID, FirstName, LastName) with their first name ascending in ascending order
5. List all female employees (ID, FirstName, LastName) with their birthdays in descending order
6. List all employees (ID, FirstName, LastName) and their department name
7. List all employees (ID, FirstName, LastName) and their manager name
8. Remove database (clean up)

Homework

H: YY: 07

Database name: University (Ref: Fundamentals of database systems)

Tables:

- Students:
 - ID: varchar(9), not null, primary
 - Name: nvarchar(50), not null
 - Class: int
 - Major: nvarchar(50)
- Courses:
 - Code: varchar (9), not null, primary
 - Name: nvarchar(50), not null, unique
 - Credit: int (greater than 0)
- Sections:

- ID: int, not null, primary
- **CourseCode**: varchar(9), not null, reference to Code of Courses table
- **Semester**: varchar(9), not null
- **Year**: int, not null
- Instructor: varchar(50), not null
- **CourseCode-Semester-Year** combination is unique
- GradeReports:
 - StudentID: varchar(9), not null, primary, reference to ID of Student stable
 - SectionID: int, not null, primary, reference to ID of Sections table
 - Grade: char(1) (A, B, C, D, E or F)
- Prerequisites:
 - CourseCode: varchar(9), not null, primary, reference to Code of Courses table
 - PrerequisiteCode: varchar(9), not null, primary, reference to Code of Courses table

Create SQL file with requirements:

1. Create database
2. Create tables
3. Add sample data

Students

ID	Name	Class	Major
17	Smith	1	CS
8	Brown	2	CS

Courses

Code	Name	Credit
CS1310	Intro to Computer Science	4
CS3320	Data Structures	4
MATH2410	Discrete Mathematics	3
CS3380	Database	3

Sections

ID	CourseCode	Semester	Year	Instructor
85	MATH2410	Fall	2007	King
92	CS1310	Fall	2007	Anderson
102	CS3320	Spring	2008	Knuth
112	MATH2410	Fall	2008	Knuth
119	CS1310	Fall	2008	Anderson
135	CS3380	Fall	2008	Stone

GradeReports

StudentID	SectionID	Grade
17	112	B
17	119	C
8	85	A
8	92	A
8	102	B
8	135	A

Prerequisites

CourseCode	PrerequisiteCode
CS3380	CS3320
CS3380	MATH2410
CS3320	CS1310

4. Find student names 'Smith'
5. List all sessions (all information) with the year in ascending order
6. List all students and grades who registered for MATH2410
7. Remove database (clean up)